Práctica Experimental
Emparejamiento Estable

Gonzalez Lugo Arlet Linette

Febrero 2026

Abstract

Este documento presenta el desarrollo experimental del algoritmo de emparejamiento estable propuesto por Gale y Shapley y su análisis de comportamiento mediante la generación de múltiples instancias del problema Stable Matching Problem (SMP) y la medición de tiempos de ejecución.

I Introducción

Se desarrollará una práctica experimental que implica la implementación y ejecución del algoritmo de emparejamiento estable propuesto por Gale y Shapley, el cual proporciona una solución formal al problema conocido como Stable Matching Problem (SMP). Asimismo, se realizará un análisis de su comportamiento y rendimiento bajo diversos conjuntos de datos y escenarios de prueba.

II Objetivo

El objetivo de esta práctica es comprobar experimentalmente el comportamiento del algoritmo de Gale y Shapley, que encuentra el emparejamiento estable entre dos conjuntos de hombres y mujeres cuya cardinalidad es N. Es decir, resolver el SMP y analizar su tiempo de ejecución conforme aumenta el tamaño del problema.

III Metodología Experimental

Esta sección describe el procedimiento experimental llevado a cabo para cumplir con el objetivo. Se utilizó el IDE Visual Studio Code como entorno de desarrollo.

III.1 Implementación del algoritmo en Python

El programa fue dividido en dos funciones principales: `main` y `gale_shapley`. En la función `gale_shapley` se implementa el algoritmo de emparejamiento estable.

La Figura 1 muestra el diagrama de flujo del algoritmo implementado.

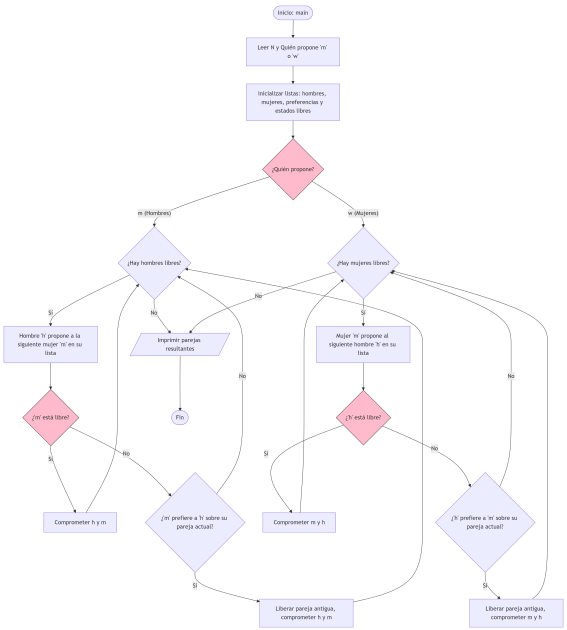

Figure 1: Diagrama de flujo del algoritmo Gale-Shapley

III.2 Generación automática de archivos de prueba

Se implementó un generador de instancias utilizando las librerías `random` y `os`. La metodología consiste en generar archivos de texto, cada uno correspondiente a un tamaño específico del problema que aumenta en múltiplos de 8 y 80.

Para cada archivo:

- Se generan identificadores desde H1 hasta Hn.
- Se generan identificadores desde M1 hasta Mn.
- Se crea una permutación aleatoria completa de mujeres y hombres mediante `random.shuffle`.

De esta manera se garantiza que cada individuo tenga una lista de preferencias completa y única.

III.3 Ejecución del algoritmo

El programa del algoritmo de Gale y Shapley se ejecutó para cada archivo generado, procesando automáticamente todas las instancias creadas.

III.4 Medición de tiempos

Se registraron los tiempos de ejecución correspondientes a cada tamaño de entrada con el objetivo de analizar experimentalmente el comportamiento asintótico del algoritmo, empleando para ello el lenguaje de programación Python.

La medición de tiempos se efectuó mediante la función `time.perf_counter()`, la cual proporciona la mayor resolución disponible para la estimación de intervalos de tiempo cortos. Cada tiempo de ejecución se expresó en milisegundos, obteniéndose dicho valor a partir del producto del tiempo medido por 1000.

Los resultados fueron almacenados en un archivo de texto (`resultados_gale_shapley.txt`), que contiene el nombre del archivo de entrada, el número de parejas y el tiempo de ejecución asociado, junto con estadísticas como los valores mínimo, máximo y promedio.

III.5 Tiempos de ejecución

A continuación se presentarán dos tablas. La primera corresponde a la Tabla 1, construida a partir de 80 archivos, cuyos valores incrementan en múltiplos de 8. La segunda tabla se obtuvo a partir de 100 archivos, con valores que incrementan en múltiplos de 80.

Table 1: Tiempos de ejecución para 80 archivos con incrementos de 8

Archivo	N	Tiempo (ms)
1	8	0.0148
2	160	0.5640
3	320	3.0848
4	480	11.8335
5	640	19.3792

En la primera tabla se usan 5 archivos con 8, 160, 320, 480 y 640 parejas. Los tiempos van de 0.0148 a 19.3792 milisegundos y se observa un incremento a partir del archivo 3, con 320 parejas.

Table 2: Tiempos de ejecución para 100 archivos con incrementos de 80

Archivo	N	Tiempo (ms)
1	80	0.1911
2	1600	118.5043
3	3200	439.2357
4	4800	961.8948
5	6400	2374.9481
6	8000	2668.5654

En la segunda tabla se usan 6 archivos con 80, 1600, 3200, 4800, 6400 y 8000 parejas.

III.6 Realización de Gráficas y su análisis

La implementación gráfica se realizó utilizando Python con la biblioteca `matplotlib`. Cada cinco archivos se realizaba un muestreo, a partir del cual se obtenían los valores (n, tiempo) para su posterior visualización.

Las gráficas incluyen un recuadro con estadísticas resumidas en la esquina superior izquierda, que muestra el número de archivos procesados, así como los tiempos mínimo, máximo y promedio.

IV Resultados

En esta sección se presentarán las gráficas y el análisis comparativo del crecimiento temporal observado respecto al tamaño N.

IV.1 Resultados para incrementos de 8

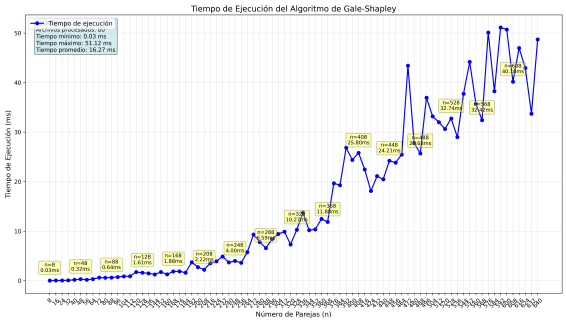

Figure 2: Gráfica lineal para 80 archivos con múltiplos de 8

En la Figura 2 se observa un crecimiento aproximadamente lineal en el tiempo de ejecución a medida que aumenta el número de parejas. Los puntos se distribuyen alrededor de una línea recta con cierta dispersión. Existe una variabilidad notable en los tiempos, especialmente para valores grandes de n, donde los puntos muestran una banda de dispersión más amplia.

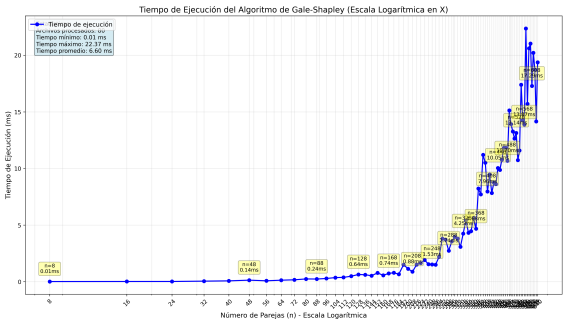

Figure 3: Gráfica semilogarítmica (eje x log) para 80 archivos con múltiplos de 8

La gráfica con escala logarítmica en el eje X (Figura 3) revela que el algoritmo de Gale-Shapley mantiene un comportamiento consistente y predecible en todo el rango analizado desde 8 hasta 640 parejas. Los puntos se distribuyen uniformemente mostrando una tendencia aproximadamente lineal, lo que sugiere una relación de tipo potencial donde el tiempo de ejecución crece de forma controlada. Los tiempos registrados van desde 0.01 ms para las instancias más pequeñas hasta 22.37 ms para las más grandes, con un promedio de 6.60 ms, lo que confirma la eficiencia práctica del algoritmo para los tamaños de problema evaluados.

IV.2 Resultados para incrementos de 80

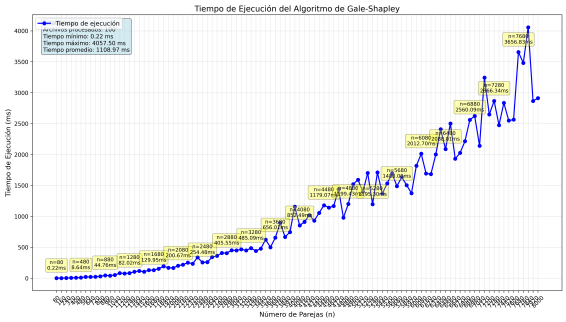

Figure 4: Gráfica lineal para 100 archivos con múltiplos de 80

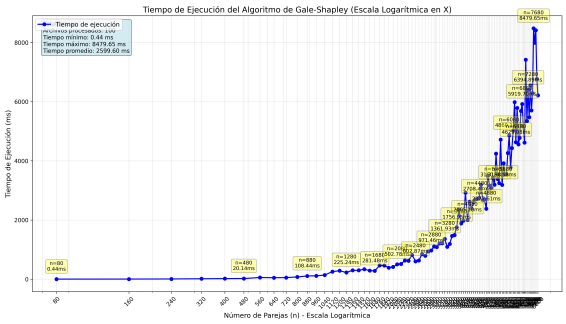

Figure 5: Gráfica semilogarítmica (eje x log) para 100 archivos con múltiplos de 80

V Conclusiones

Como podemos observar en las Figuras 3 y 5 con escala logarítmica en el eje X, se revela de forma contundente el comportamiento asintótico del algoritmo de Gale-Shapley. Particularmente en la Figura 5, para instancias de gran tamaño, se observa un crecimiento más que lineal en el tiempo de ejecución a medida que aumenta el número de parejas, evidenciado por el incremento drástico desde 0.44 ms para las instancias más pequeñas hasta 8479.65 ms para las de mayor tamaño.

Esta tendencia ascendente y no lineal es consistente con la complejidad teórica del algoritmo, que es $O(n^2)$ en el peor caso. La escala logarítmica en el eje de las abscisas permite apreciar que, aunque el crecimiento no es exponencial, el tiempo de ejecución se incrementa de manera significativa y sostenida, lo que evidencia la dependencia cuadrática del algoritmo respecto al tamaño de la entrada. Este comportamiento valida experimentalmente las predicciones teóricas y demuestra que, para instancias de gran escala, el costo computacional puede volverse considerable, lo cual es fundamental considerar en aplicaciones prácticas que requieran emparejamientos estables con un número elevado de participantes.